

Four RSA Attacks: Håstad, Common Modulus, Cycling, and Bleichenbacher

Each attack below reuses a tool already established in Lesson 1 — the Chinese Remainder Theorem, the Extended Euclidean Algorithm, Euler’s theorem, and RSA’s multiplicative malleability, respectively — now applied as an attack tool rather than a construction (or correctness) tool.

1. Håstad’s Broadcast Attack

When this arises. A single plaintext must reach several independent recipients — for example, a vendor distributing the same encrypted license key or firmware update to many customers, or an encrypted group announcement. Each recipient has generated their own RSA key pair entirely independently of the others; no key is shared. What makes the attack possible is that many of them use the same small public exponent (historically $e = 3$, chosen purely for fast encryption on constrained hardware) as a common convention, while the sender encrypts the identical message separately under each recipient’s own public key.

Setup. Small public exponent (e.g. $e = 3$), same message m broadcast to 3 recipients with independent, pairwise-coprime moduli n_1, n_2, n_3 :

$$c_i = m^3 \bmod n_i, \quad i = 1, 2, 3.$$

Core insight. Three small, separately-opaque worlds ($\bmod n_1$, $\bmod n_2$, $\bmod n_3$) get stitched by CRT into one big world ($\bmod N = n_1 n_2 n_3$) — and against that much bigger modulus, m^3 might not wrap around at all. It is the “no modular wraparound” flaw from Lesson 1, resurfacing one level up.

Theorem 1. *If $m < \min(n_1, n_2, n_3)$, an eavesdropper who sees c_1, c_2, c_3 recovers m — no private key needed.*

Proof.

- CRT combines $x \equiv c_i \pmod{n_i}$ ($i = 1, 2, 3$) into a *unique* $C \bmod N = n_1 n_2 n_3$, computed directly from public data.
- m^3 also satisfies all three congruences (by definition of c_i), so by CRT’s uniqueness, $C \equiv m^3 \pmod{N}$.
- Since $m < \min n_i$, we get $m^3 < N$, so C and m^3 are both nonnegative integers $< N$ that agree $\bmod N$ — they are *exactly equal* as integers, not just congruent.
- Take an ordinary integer cube root of C .

Worked example. $n_1 = 143 = 11 \cdot 13$, $n_2 = 323 = 17 \cdot 19$, $n_3 = 667 = 23 \cdot 29$ (pairwise coprime), $m = 17$:

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

$N = n_1 n_2 n_3 = 30,808,063$. CRT-combining $(51, 68, 244)$ gives $C = 4913$. Since $17^3 = 4913 < N$, there was no wraparound, so C is the exact integer m^3 . Then $\sqrt[3]{4913} = 17 = m$, recovered with no private key.

Practical consequence. Never send the same or related message to multiple recipients under a small e without randomized padding; OAEP (Lesson 3) fixes this.

2. Common Modulus Attack

When this arises. A central authority tries to avoid the expensive step of generating fresh large primes for every user. Since primality testing dominates the cost of RSA key generation while computing a new exponent pair from an already-known factorization is comparatively cheap, an authority might generate a single modulus $n = pq$ once and then issue each user a distinct exponent pair (e_i, d_i) over that same shared n . This was a real anti-pattern in some early multi-user and embedded key-management systems, where saving the primality-search cost was mistaken for a safe optimization.

Setup. One shared modulus n , two users with coprime exponents e_1, e_2 ($\gcd(e_1, e_2) = 1$), same message encrypted under both:

$$c_1 = m^{e_1} \bmod n, \quad c_2 = m^{e_2} \bmod n.$$

Core insight. $\gcd(e_1, e_2) = 1$ means the Extended Euclidean Algorithm can write $1 = ae_1 + be_2$ — the same computation used to build the private key d . Here it is aimed at the exponents directly: combining c_1, c_2 with exponents a, b combines m^{e_1}, m^{e_2} into $m^1 = m$.

Theorem 2. If $\gcd(e_1, e_2) = 1$, anyone knowing only (n, e_1, e_2, c_1, c_2) recovers m — no private key needed.

Proof.

- Extended Euclid on (e_1, e_2) gives integers a, b with $ae_1 + be_2 = 1$.
- $c_1^a c_2^b \equiv (m^{e_1})^a (m^{e_2})^b \equiv m^{ae_1 + be_2} \equiv m^1 \equiv m \pmod{n}$, by the exponent laws alone.
- If (say) $b < 0$: compute $c_2^{-1} \bmod n$ first (Extended Euclid again), then raise that to $|b|$.

Worked example. $n = 3233$ ($p = 61, q = 53$), $e_1 = 17, e_2 = 7, m = 42$:

$$c_1 = 42^{17} \bmod 3233 = 2557, \quad c_2 = 42^7 \bmod 3233 = 240.$$

Extended Euclid(17, 7) gives $a = -2, b = 5$ (check: $-2 \cdot 17 + 5 \cdot 7 = 1$). Since $a < 0$: $c_1^{-1} \bmod n = 3123$. Then

$$m = (c_1^{-1})^2 \cdot c_2^5 \bmod n = 42.$$

Practical consequence. Never share a modulus across users or purposes, even with different exponents — it is broken the instant any two exponents on it are coprime, which is the common case.

3. Cycling Attack (Conceptual)

When this arises. Unlike the two attacks above, this one requires no mistake in how keys are generated or used — it applies, in principle, to any RSA ciphertext at all. It is not a practical

threat in deployed systems (the cycle length is normally far too large to compute), but it is the cleanest illustration, using only tools already introduced, of why key-generation guidance has historically recommended avoiding moduli where $\lambda(n)$ has only small prime factors.

Setup. Only (n, e, c) with $c = m^e \bmod n$. Repeatedly re-encrypt: $c_1 = c^e \bmod n$, $c_2 = c_1^e \bmod n, \dots$ until the sequence returns to c .

Core insight. This is Euler’s theorem, one level up. Repeatedly multiplying by a cycles back to 1 after $\varphi(n)$ steps; repeatedly *encrypting* (exponentiating by e) must also cycle back eventually, since encryption is invertible. The plaintext sits in plain sight, one step before the cycle closes.

Theorem 3. *Let k be the smallest positive integer with $c_k = c$. Then $m = c_{k-1}$.*

Proof.

- By induction, $c_i \equiv m^{e^{i+1}} \pmod{n}$.
- By definition of k : $m^{e^{k+1}} \equiv m^e \pmod{n}$, i.e. $e^{k+1} \equiv e \pmod{\text{ord}_n(m)}$.
- e is coprime to $\varphi(n)$ by construction, hence coprime to $\text{ord}_n(m) \mid \varphi(n)$ too, so one factor of e cancels: $e^k \equiv 1 \pmod{\text{ord}_n(m)}$.
- So $c_{k-1} \equiv m^{e^k} \equiv m^1 \equiv m \pmod{n}$ — one full lap plus one step back, the same picture as Euler’s theorem in Lesson 1.

Worked example. $p = 47, q = 59 \Rightarrow n = 2773$, $\varphi(n) = 2668$, $e = 17$, $m = 123$:

$$c = 123^{17} \bmod 2773 = 1924.$$

Repeatedly raising to the 17th power, after 44 iterations the sequence returns to $c_{44} = 1924 = c$. One step earlier, $c_{43} = 123 = m$.

Why it stays conceptual. At real key sizes the cycle length is astronomically large and the attack is infeasible in practice. It demonstrates that RSA encryption is iteration inside a finite group, and previews why $\lambda(n)$ having a large prime factor matters — the same concern as avoiding smooth $p - 1$ (Lesson 1).

4. Bleichenbacher’s Padding Oracle Attack

When this arises. A server decrypts RSA ciphertexts that were formatted with padding before encryption, and — through an error message, a connection reset, or even just a difference in response time — reveals whether the padding was valid, without intending to disclose anything else. This is remarkably easy to do by accident: TLS servers implementing the older PKCS#1 v1.5 padding standard routinely returned a distinguishable “bad padding” alert. This is not a historical curiosity: refinements of the same attack (Bardou et al., 2012; the ROBOT attack, 2017) found that major production web servers, including large commercial services, were still exploitable through some variant of this technique nearly twenty years after it was first published. It remains one of the most consequential vulnerabilities ever found in a widely deployed cryptographic protocol.

Setup. PKCS#1 v1.5 formats a k -byte plaintext block as

$$\text{EB} = 0x00 \parallel 0x02 \parallel \text{PS} \parallel 0x00 \parallel D,$$

where D is the message and PS is a string of non-zero random padding bytes filling the block out to exactly k bytes. A *padding oracle* is any mechanism that lets an attacker submit a chosen ciphertext c' and learn one bit: does $(c')^d \bmod n$ decrypt to a block starting with $0x00 \ 0x02$?

Example block, $k = 8$: message $D = 0x1234$, padding $PS = 0xA1\ 0x7F\ 0x3C$, gives $EB = 0x00\ 0x02\ 0xA1\ 0x7F\ 0x3C\ 0x00\ 0x1234$ (exactly 8 bytes).

Core insight. The attack combines two facts, both already familiar:

- *Malleability* (Lesson 1, and the same operation used in the Common Modulus attack above): given $c = m^e \bmod n$, anyone can compute $c \cdot s^e \bmod n$ for a chosen s , which is a valid encryption of $m \cdot s \bmod n$ — no secret required.
- *The oracle is a narrow-range membership test.* Since valid padding means the decrypted integer lies in a specific small range $[2B, 3B)$ (where $B = 2^{8(k-2)}$), a YES answer to the oracle on $c \cdot s^e \bmod n$ reveals: $m \cdot s \bmod n$ *currently lies in* $[2B, 3B)$ — a strong constraint on the unknown m , for an s the attacker chose and knows.

Repeating this with carefully chosen values of s , each new YES answer shrinks the set of values consistent with everything learned so far, narrowing down m step by step.

- **Step 1 (blinding search):** find the smallest s_1 such that $c \cdot s_1^e \bmod n$ passes the oracle.
- **Step 2 (interval narrowing):** maintain a shrinking set of intervals guaranteed to contain m ; each query on a new s_i either confirms an interval or splits it further, using the known range $[2B, 3B)$.
- **Step 3 (interval refinement):** once a single interval remains, search increasingly precise values of s to shrink it toward one point.
- **Return** the unique m remaining once the interval has width 1.

Practical consequence. Never let decryption-failure reasons be distinguishable to an attacker, whether through error content, timing, or connection behavior. The complete fix is to stop using PKCS#1 v1.5 for new designs and use OAEP (Lesson 3) instead, whose security proof directly rules out this entire class of attack.

All four attacks above reuse tools from Lesson 1 — CRT, the Extended Euclidean Algorithm, Euler’s theorem, and RSA’s multiplicative malleability — as attack tools rather than construction tools.